

WorkSpace and ROS Topic

Prof.: Dr. Oybek Eraliev

TA: Jasurbek Mamurov



Learning Objectives:

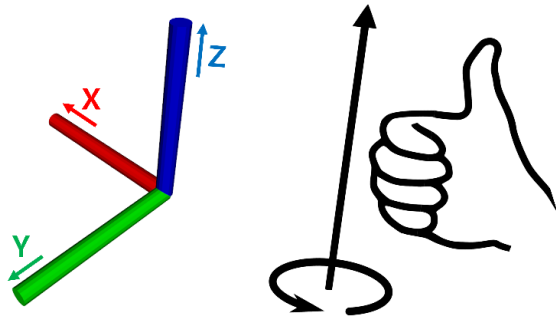
- Install and configure VSCode for ROS development
- Understand workspace creation with *catkin* and *roscpp*
- Master commonly used ROS commands
- Create a custom ROS package

Content:

- VSCode Installation & ROS Extensions
- Workspace Creation
- Commonly Used ROS Commands
- Creating a Custom Package

Things to know before programming ROS

- **Standard unit**
 - SI unit
- **Coordinate representation**
 - x: forward, y: left, z: up
 - Right-hand rule



Quantity	Unit
angle	radian
frequency	hertz
force	newton
power	watt
voltage	volt

Quantity	Unit
length	meter
mass	kilogram
time	second
current	ampere
temperature	celsius

Things to know before programming ROS

- Programing rules

Object	Naming rule	example
Package	under_scored	Ex) first_ros_package
Topic, service	under_scored	Ex) raw_image
File	under_scored	Ex) turtlebot3_fake.cpp
Namespace	under_scored	Ex) ros_awesome_package
Variable	under_scored	Ex) string table_name;
type	CamelCased	Ex) typedef int32_t PropertiesNumber;
Class	CamelCased	Ex) class UriTable
Structure	CamelCased	Ex) struct UriTableProperties
Enumeration type	CamelCased	Ex) enum ChoiceNumber
Function	camelCased	Ex) addTableEntry();
Method	camelCased	Ex) void setNumEntries(int32_t num_entries)
Constant	ALL_CAPITALS	Ex) const uint8_t DAYS_IN_A_WEEK = 7;
Macro	ALL_CAPITALS	Ex) #define PI_ROUNDED 3.0

VSCode Installation & ROS Extensions

Installation Steps

Walk students through:

1. Download VSCode from <https://code.visualstudio.com>
2. Install the following extensions:
 - **ROS** (Microsoft)
 - **Python** (Microsoft)
 - **CMake Tools**
 - **C/C++** (useful for later)

VSCode Installation & ROS Extensions

Download & Install the .deb package (Recommended for Ubuntu/Debian)

Download the latest VSCode .deb package

```
wget -O vscode.deb "https://code.visualstudio.com/sha/download?build=stable&os=linux-deb-x64"
```

Install it

```
sudo dpkg -i vscode.deb
```

Fix any dependency issues if needed

```
sudo apt --fix-broken install
```

Clean up

```
rm vscode.deb
```

VSCode Installation & ROS Extensions

Add Microsoft's Official Repository (Best for auto-updates)

Install dependencies

```
sudo apt install wget gpg
```

Add Microsoft GPG key

```
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor > packages.microsoft.gpg
```

```
sudo install -o root -g root -m 644 packages.microsoft.gpg /etc/apt/trusted.gpg.d/
```

```
rm packages.microsoft.gpg
```

Add the VSCode repository

```
sudo sh -c 'echo "deb [arch=amd64] https://packages.microsoft.com/repos/code stable main" > /etc/apt/sources.list.d/vscode.list'
```

Update and install

```
sudo apt update
```

```
sudo apt install code
```


VSCode Installation & ROS Extensions

For MacBook M series products

- `wget -O vscode.deb "https://code.visualstudio.com/sha/download?build=stable&os=linux-deb-arm64"`
- `sudo dpkg -i vscode.deb`
- `sudo apt --fix-broken install`
- `rm vscode.deb`

VSCode Installation & ROS Extensions

Snap (Simplest single command)

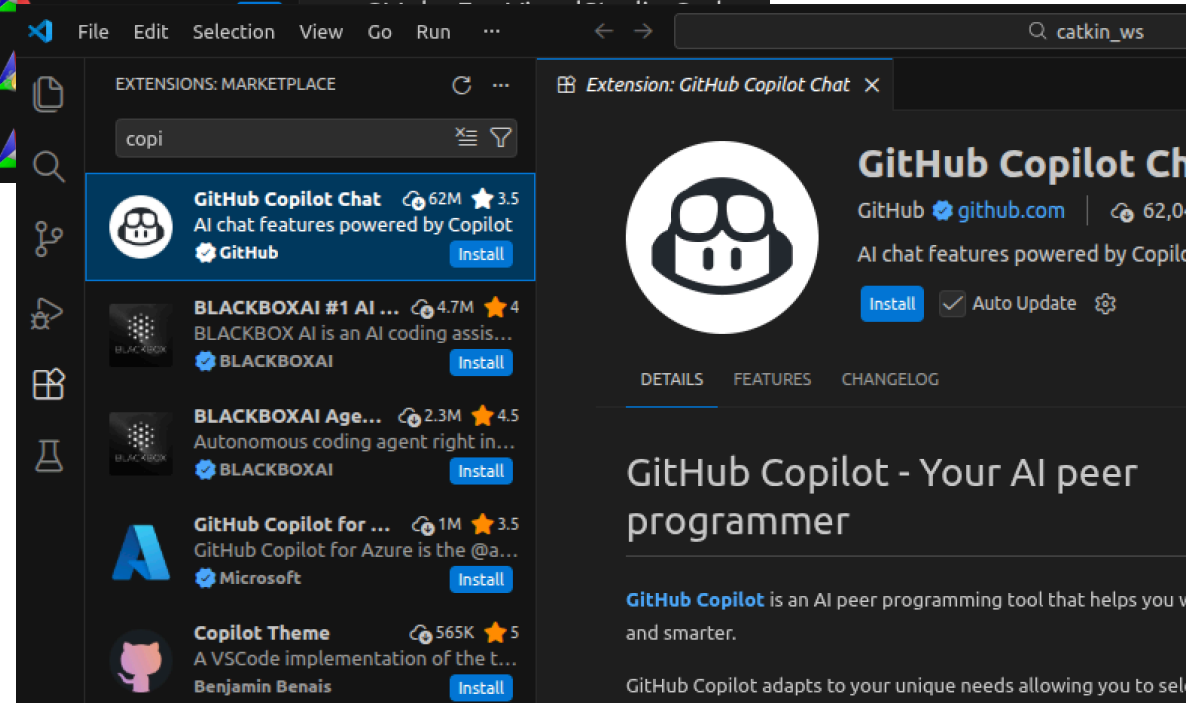
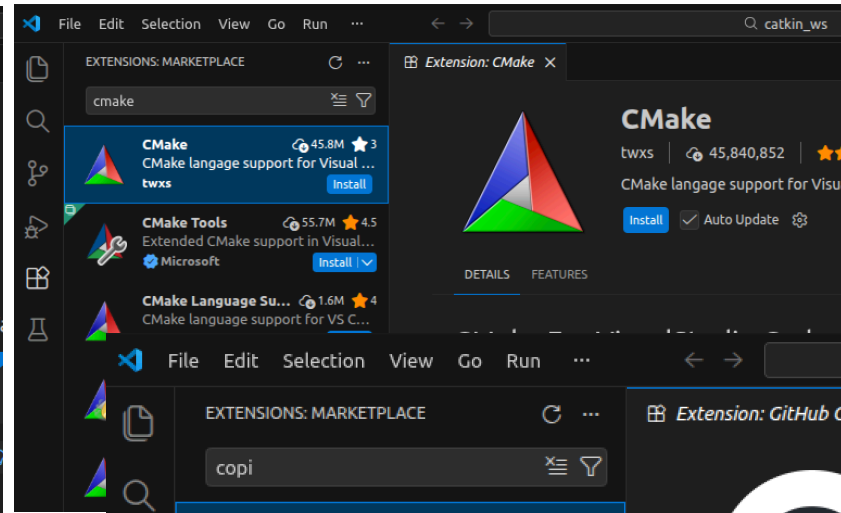
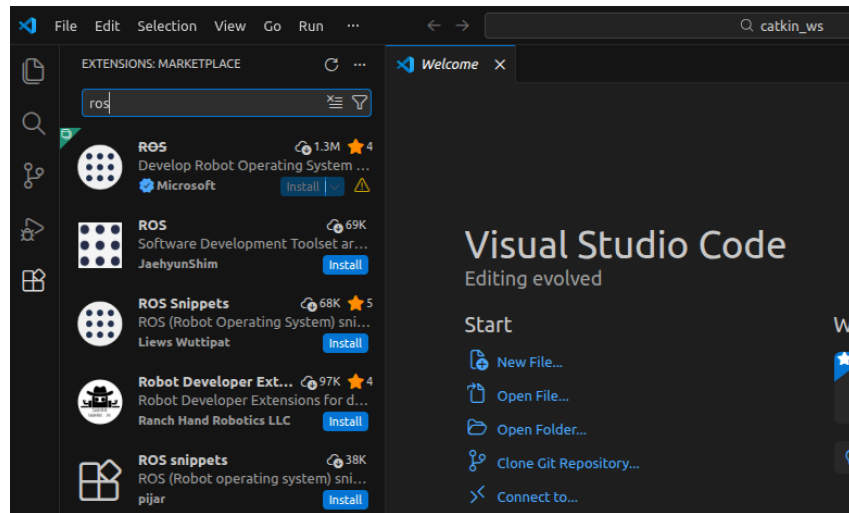
```
sudo apt install snap  
sudo snap install code --classic
```

Launch VSCode from terminal

Once installed, you can open it directly from the terminal which is very handy for ROS:

```
code .                # Open current directory as workspace  
code ~/catkin_ws      # Open your catkin workspace directly
```

VSCode Installation & ROS Extensions



Content:

- ~~VSCode Installation & ROS Extensions~~
- Workspace Creation
- Commonly Used ROS Commands
- Creating a Custom Package

Workspace Creation

- Catkin Workspace (Modern — Recommended)

```
# Create workspace directories
```

```
mkdir -p ~/catkin_ws/src
```

```
cd ~/catkin_ws/
```

```
# Initialize and build the workspace
```

```
catkin_make
```

```
# Source the workspace (add this to ~/.bashrc)
```

```
source ~/catkin_ws/devel/setup.bash
```

```
echo "source ~/catkin_ws/devel/setup.bash" >> ~/.bashrc
```

```
gedit ~/.bashrc
```

Workspace Creation

- **Rosbuild Workspace (Legacy — for context)**

```
# Create workspace
```

```
mkdir ~/rosbuild_ws
```

```
cd ~/rosbuild_ws
```

```
rosws init . /opt/ros/noetic
```

```
# Source it
```

```
source ~/rosbuild_ws/setup.bash
```

Workspace Creation

- Instructor note: Explain that ***roscbuild*** is legacy. Most modern ROS 1 projects use ***catkin***.

Catkin vs Rosbuild Comparison Table

Feature	catkin	roscbuild
Build system	CMake-based	Custom scripts
ROS version	ROS Groovy+	Pre-Groovy
Python support	Native	Limited
Recommended	 Yes	 Legacy

Workspace Creation

- ~~VSCode Installation & ROS Extensions~~
- ~~Workspace Creation~~
- Commonly Used ROS Commands
- Creating a Custom Package

ROS execution command

command	importance	Command description	Detailed description
roscd	★★★	roscd(changes directory)	Move to the directory of the specified ROS package
rosls	★☆☆	rosls(lists files)	Check the list of files in the ROS package
roscd	★☆☆	roscd(editor)	Editing files in the ROS package
roscp	★☆☆	roscp(copies files)	Copying Files in the ROS Package
rospd	☆☆☆	rospushd	Adding Directories to the ROS Directory Index
roscd	☆☆☆	roscd(directory)	Check the ROS directory index

ROS execution command

command	importance	Command description	Detailed description
roscore	★★★	roscore	- master (ROS name service) - rosout (log record) - parameter server (parameter management)
roslaunch	★★★	roslaunch	Running the Node
rosclean	★★☆	rosclean	Running multiple nodes with options
			Scan or delete ROS log files

ROS information command

command	importance	Command description	Detailed description
rostopic	★★★	rostopic	Check ROS topic information
rosservice	★★★	rosservice	Check ROS service information
roscall	★★★	roscall	Check ROS node information
rosparam	★★★	rosparam(parameter)	Check and modify ROS parameter information
rosbag	★★★	rosbag	ROS message recording, playback
rosmmsg	★★☆	rosmmsg	Check ROS message information
rossrv	★★☆	rossrv	Check ROS service information
rosverison	★☆☆	rosverison	Determining ROS Package and Deployment Release Version Information
roswtf	☆☆☆	roswtf	ROS System Inspection

ROS catkin command

command	importance	Detailed description
catkin_create_pkg	★★★	Automatic generation of packages
catkin_make	★★★	Builds based on the Catkin build system
catkin_eclipse	★★☆	Changes made by Eclipse to the package created by the Catkin build system
catkin_prepare_release	★★☆	Log cleanup and version tagging used for release
catkin_generate_changelog	★★☆	Creating or Updating the CHANGELOG.rst File When Releasing
catkin_init_workspace	★★☆	Initializing the Catkin build system's working folder
catkin_find	★☆☆	Search catkin

ROS package command

command	importance	Command description	Detailed description
rospack	★★★	rospack(age)	See information related to the ROS package
roinstall	★★☆	roinstall	Installing the ROS Additional Package
roscdep	★★☆	roscdep(endencies)	Install the package's dependency files
rosllocate	☆☆☆	rosllocate	ROS package information Related commands
roscrcreate-pkg	☆☆☆	roscrcreate-pkg	Automatically generate ROS package (used in old rosbuilt system)
rosmake	☆☆☆	rosmake	Build the ROS package (formerly used by the rosbuilt system)

Commonly Used ROS Commands

Core System

<code>roscore</code>	<code># Start ROS master</code>
<code>rostopic list</code>	<code># List active nodes</code>
<code>rostopic info /node_name</code>	<code># Node details</code>
<code>rostopic kill /node_name</code>	<code># Kill a node</code>

Topics

<code>rostopic list</code>	<code># List all topics</code>
<code>rostopic echo /topic_name</code>	<code># Print topic messages live</code>
<code>rostopic info /topic_name</code>	<code># Topic type and connections</code>
<code>rostopic pub /topic std_msgs/String "data: 'hello'"</code>	<code># Publish manually</code>
<code>rostopic hz /topic_name</code>	<code># Check publish rate</code>

Commonly Used ROS Commands

Packages & Nodes

```
rospack list           # List all packages
rospack find package_name # Find package path
roslaunch package_name node_name # Run a node
roslaunch package_name file.launch # Launch file
```

Introspection Tools

```
rqt_graph      # Visualize node/topic graph
rqt_console    # View log messages
rviz           # 3D visualization
rosmmsg show std_msgs/String # Show message structure
```

Content:

- ~~VSCode Installation & ROS Extensions~~
- ~~Workspace Creation~~
- ~~Commonly Used ROS Commands~~
- Creating a Custom Package

Creating a Custom Package

1. Creating the package

```
$ cd ~/catkin_ws/src
$ catkin_init_workspace
$ catkin_create_pkg ros_tutorials_topic message_generation std_msgs
roscpp
$ cd ros_tutorials_topic
$ ls
include          → header file folder
src              → source code folder
CMakeLists.txt   → build configuration file
package.xml      → package configuration file
$ cd ~/catkin_ws/ && catkin_make
$ source ~/catkin_ws/devel/setup.bash
```

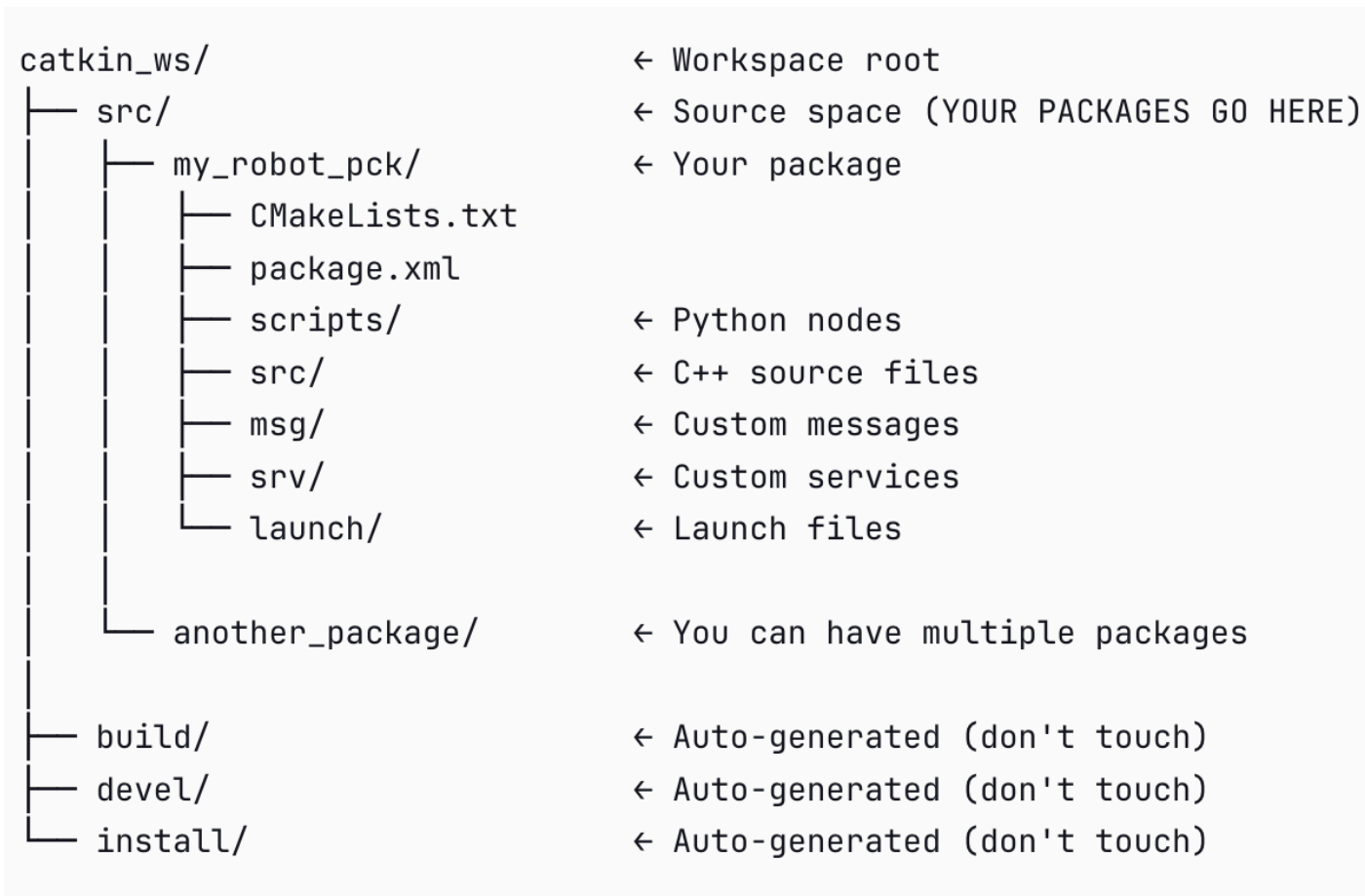
Creating a Custom Package

Package Structure

```
my_robot_pkg/  
├── CMakeLists.txt      # Build rules  
├── package.xml         # Package metadata & dependencies  
├── src/               # C++ source files  
├── scripts/           # Python nodes (we will use this)  
└── msg/               # Custom message definitions (Class 2)
```

Creating a Custom Package

Workspace Structure



Creating a Custom Package

package.xml Walkthrough

```
<?xml version="1.0"?>
<package format="2">
  <name>my_robot_pkg</name>
  <version>0.0.1</version>
  <description>My first ROS package</description>

  <maintainer email="student@university.edu">Student</maintainer>
  <license>MIT</license>

  <buildtool_depend>catkin</buildtool_depend>
  <depend>rospy</depend>
  <depend>std_msgs</depend>
</package>
```

Creating a Custom Package

Installation of additional dependencies to the package

Method 1: Manually Edit `package.xml` and `CMakeLists.txt`

Step 1 — Edit `package.xml`

```
<?xml version="1.0"?>
<package format="2">
  <name>my_package</name>
  <version>0.0.1</version>
  <description>My ROS package</description>
  <maintainer email="student@uni.edu">Student</maintainer>
  <license>MIT</license>

  <buildtool_depend>catkin</buildtool_depend>

  <!-- ADD YOUR DEPENDENCIES HERE -->
  <depend>rospy</depend>
  <depend>roscpp</depend>
  <depend>std_msgs</depend>
  <build_depend>message_generation</build_depend>
  <exec_depend>message_runtime</exec_depend>

</package>
```

Creating a Custom Package

Installation of additional dependencies to the package

Method 1: Manually Edit `package.xml` and `CMakeLists.txt`

Step 1 — Edit `package.xml`

<depend> is the simplest — it means "I need this both to build AND to run." It's a shortcut that covers both build and runtime. So writing `<depend>rospy</depend>` is equivalent to writing both `<build_depend>rospy</build_depend>` and `<exec_depend>rospy</exec_depend>` together. You use this for straightforward libraries like `rospy`, `roscpp`, and `std_msgs` that you need at all stages.

<build_depend> means "I only need this when compiling/building my package, not at runtime." In your case, `message_generation` is a tool that generates C++/Python code from your `.msg` files during the build process. Once your package is compiled, `message_generation` is no longer needed — it's like a compiler you only need during construction.

<exec_depend> means "I only need this when the package is actually running, not during build." `message_runtime` is the counterpart to `message_generation` — it provides the libraries needed to actually use those generated messages while your nodes are executing.

Creating a Custom Package

Installation of additional dependencies to the package

Method 1: Manually Edit `package.xml` and `CMakeLists.txt`

Step 2 — Edit `CMakeLists.txt`

```
find_package(catkin REQUIRED COMPONENTS
  rospy
  roscpp
  std_msgs
  message_generation    # add here
)
```

Creating a Custom Package

Installation of additional dependencies to the package

Method 1: Manually Edit `package.xml` and `CMakeLists.txt`

Step 3 — Install the dependency on the system and rebuild

```
sudo apt update
sudo apt install ros-noetic-rospy
sudo apt install ros-noetic-roscpp
sudo apt install ros-noetic-std-msgs
```

```
cd ~/catkin_ws
catkin_make
source devel/setup.bash
```


Creating a Custom Package

Installation of additional dependencies to the package

Method 2: `rosdep` — Automatic Dependency Installer (Most Practical)

This is the professional way. `rosdep` reads your `package.xml` and installs everything automatically.

```
# First time setup (only once per system)
sudo rosdep init
rosdep update
```

```
# Navigate to your workspace
cd ~/catkin_ws
```

```
# Install all dependencies for all packages in src/
rosdep install --from-paths src --ignore-src -r -y
```

Creating a Custom Package

Installation of additional dependencies to the package

Method 2: `rosdep` — Automatic Dependency Installer (Most Practical)

This is the professional way. `rosdep` reads your `package.xml` and installs everything automatically.

```
# Install all dependencies for all packages in src/  
rosdep install --from-paths src --ignore-src -r -y
```

Flag	Meaning
<code>--from-paths src</code>	Look inside the src folder
<code>--ignore-src</code>	Skip packages that are already in your workspace
<code>-r</code>	Continue even if some deps are missing
<code>-y</code>	Auto-confirm all installs

Creating a Custom Package

Checking installed dependencies

```
# Show all dependencies (direct + indirect)
```

```
rospack depends my_package
```

```
# Show only DIRECT dependencies (first level only)
```

```
rospack depends1 my_package
```

```
# Returns raw package manifest info
```

```
rospack find my_package
```

```
# Find where the package lives
```

```
rospack list
```

```
# List all packages and paths
```

Creating a Custom Package

Checking installed dependencies

If checking method doesn't work install python3-rosdep as follows:

```
# Method 1: via apt (Recommended)
sudo apt install python3-rosdep
```

```
# Then initialize
sudo rosdep init
rosdep update
```

CHECKLIST

- ☒ 1. Start roscore in one terminal. List all active nodes. What do you see?
- ☐ 2. In a new terminal, run: `roslaunch turtlesim turtlesim_node` → What is the full name of this node?
- ☐ 3. List all active topics. How many are there?
- ☐ 4. Echo the `/turtle1/pose` topic. What fields does it contain?
- ☐ 5. What is the message type of `/turtle1/cmd_vel`?
- ☐ 6. Manually publish to `/turtle1/cmd_vel` to make the turtle move.
- ☐ 7. Open `rqt_graph`. Sketch the node/topic graph on paper.
- ☐ 8. Create a package called 'my_first_pkg' with `roscpp` and `std_msgs`.
- ☐ 9. Find the path of the `turtlesim` package using `rospack`.
- ☐ 10. Check the publish rate (hz) of `/turtle1/pose`.



Thank you!

